



ΠΟΛΥΤΕΧΝΕΙΟ ΚΡΗΤΗΣ

Τηλεπικοινωνιακά Συστήματα Ι

Άσκηση 3

Ημερομηνία Παράδοσης: 12 Ιουνίου 2023

Η εργασία μπορεί να παραδοθεί από ομάδες $\leq$ δύο ατόμων

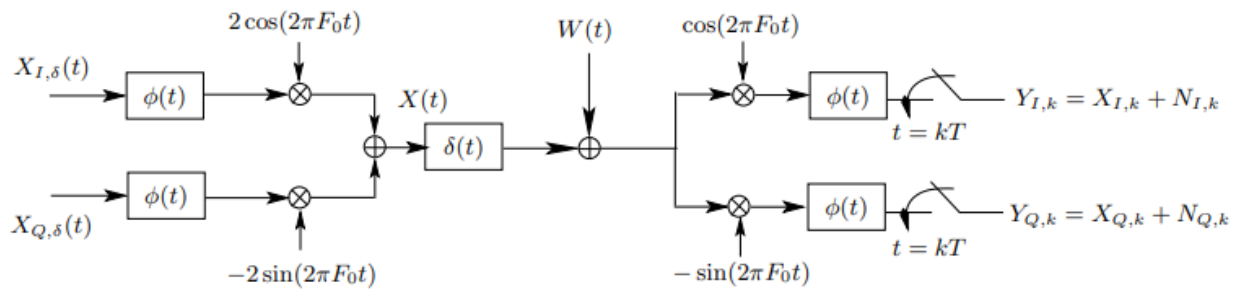
Διδάσκων: Αθανάσιος Π. Λιάβας

Μονάδες 130/300

Ομάδα Χρηστών 52

	Φοιτητής 1	Φοιτητής 2
Επώνυμο	Τσικριτζάκης	Κουτσοβασίλης
Όνομα	Γεώργιος-Μάριος	Βασίλειος
A.M.	2020030055	2020030137

Ώρες Ενασχόλησης : 17 ώρες



Σε αυτή την άσκηση, θα προσομοιώσουμε το τηλεπικοινωνιακό σύστημα του Σχήματος, υποθέτοντας ότι χρησιμοποιείται διαμόρφωση 16-PSK, και θα μελετήσουμε την απόδοσή του.

1.

Για δεδομένο N (ενδεικτικά, $N = 100$), να δημιουργήσετε δυαδική ακολουθία `bit_seq` με στοιχεία $4N$ ισοπίθανα bits.

Όπως και στις προηγούμενες ασκήσεις τα ισοπίθανα bits δημιουργήθηκαν χρησιμοποιώντας την συνάρτηση `randn` του MATLAB.

```
%%1
%%Create random bits:
N=100;
bit_seq = (sign(randn(4*N, 1)) + 1)/2;
```

2.

(15) Να γράψετε συνάρτηση function `X = bits to PSK 16(bit_seq)` η οποία, χρησιμοποιώντας κωδικοποίηση Gray, απεικονίζει τη δυαδική ακολουθία εισόδου `bit_seq` σε ακολουθία 16-PSK συμβόλων X , μήκους N , με στοιχεία τα δισδιάστατα διανύσματα :

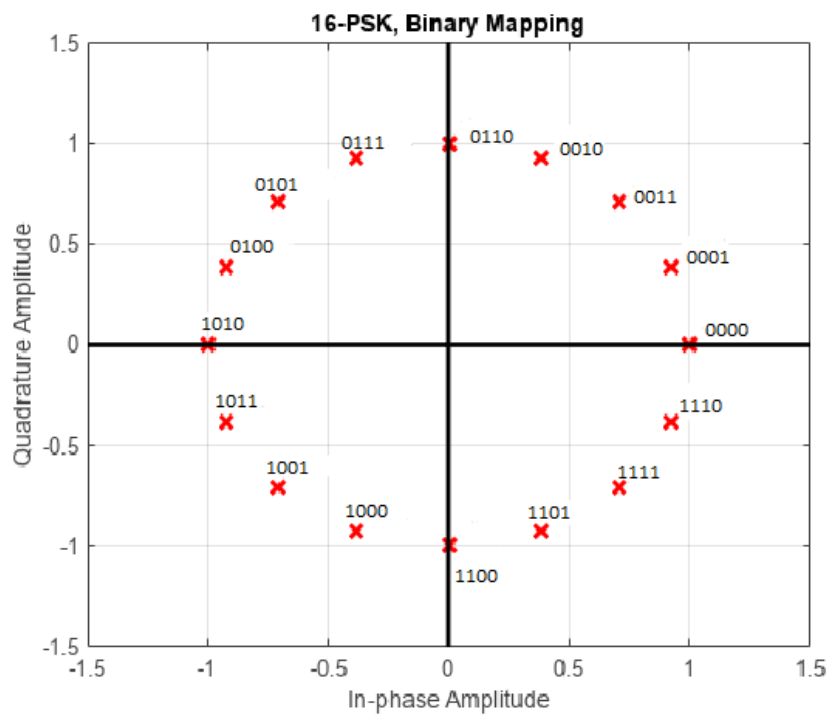
$$X_n = \begin{bmatrix} X_{I,n} \\ X_{Q,n} \end{bmatrix} \text{ για } n = 0, \dots, N-1$$

Κάθε διάνυσμα X_n , για $n = 0, \dots, N-1$, παίρνει τιμές από το αλφάβητο 16-PSK $\{x_0, \dots, x_{15}\}$ με

$$X_m = \begin{bmatrix} \cos\left(\frac{2\pi m}{16}\right) \\ \sin\left(\frac{2\pi m}{16}\right) \end{bmatrix} \text{ για } m = 0, \dots, N-1$$

Χρησιμοποιήσαμε την εξής κωδικοποίηση Gray:

```
%%2
%%Map these bits to 16 psk with gray encoding:
% 0000 -> 0 radians      X0
% 0001 ->  $\pi/8$  radians  X1
% 0011 ->  $\pi/4$  radians  X2
% 0010 ->  $3\pi/8$  radians X3
% 0110 ->  $\pi/2$  radians  X4
% 0111 ->  $5\pi/8$  radians X5
% 0101 ->  $3\pi/4$  radians X6
% 0100 ->  $7\pi/8$  radians X7
% 1010 ->  $\pi$  radians    X8
% 1011 ->  $9\pi/8$  radians X9
% 1001 ->  $5\pi/4$  radians X10
% 1000 ->  $11\pi/8$  radians X11
% 1100 ->  $3\pi/2$  radians X12
% 1101 ->  $13\pi/8$  radians X13
% 1111 ->  $7\pi/4$  radians X14
% 1110 ->  $15\pi/8$  radians X15
```



Και ο κώδικας της συνάρτησης:

```
function X = bits_to_PSK_16(bit_seq)
    num_symbols = floor(length(bit_seq)/4); % Number of 16PSK symbols

    X = zeros(2, num_symbols); % Initialize symbol matrix

    for i = 1:4:length(bit_seq)
        bits = bit_seq(i:i+3);

        if bits(1) == 0
            if bits(2) == 0
                if bits(3) == 0
                    if bits(4) == 0
                        % 0000 => X0
                        X(:, (i+3)/4) = [cos(2*pi*0/16); sin(2*pi*0/16)];
                    else
                        % 0001 => X1
                        X(:, (i+3)/4) = [cos(2*pi*1/16); sin(2*pi*1/16)];
                    end
                else
                    if bits(4) == 1
                        % 0011 => X2
                        X(:, (i+3)/4) = [cos(2*pi*2/16); sin(2*pi*2/16)];
                    else
                        % 0010 => X3
                        X(:, (i+3)/4) = [cos(2*pi*3/16); sin(2*pi*3/16)];
                    end
                end
            else %bit2=1
                if bits(3) == 1
                    if bits(4) == 0
                        % 0110 => X4
                        X(:, (i+3)/4) = [cos(2*pi*4/16); sin(2*pi*4/16)];
                    else
                        % 0111 => X5
                        X(:, (i+3)/4) = [cos(2*pi*5/16); sin(2*pi*5/16)];
                    end
                else
                    if bits(4) == 1
                        % 0101 => X6
                        X(:, (i+3)/4) = [cos(2*pi*6/16); sin(2*pi*6/16)];
                    else
                        % 0100 => X7
                        X(:, (i+3)/4) = [cos(2*pi*7/16); sin(2*pi*7/16)];
                    end
                end
            end
        else %bit1 = 1
            if bits(2) == 1
                if bits(3) == 0
                    if bits(4) == 0
                        % 1100 => X12
                        X(:, (i+3)/4) = [cos(2*pi*12/16); sin(2*pi*12/16)];
                    else
                        % 1101 => X13
                        X(:, (i+3)/4) = [cos(2*pi*13/16); sin(2*pi*13/16)];
                    end
                else %bit3 = 1
                    if bits(4) == 0
                        % 1000 => X8
                        X(:, (i+3)/4) = [cos(2*pi*8/16); sin(2*pi*8/16)];
                    else
                        % 1001 => X9
                        X(:, (i+3)/4) = [cos(2*pi*9/16); sin(2*pi*9/16)];
                    end
                end
            else
                if bits(4) == 0
                    % 1010 => X10
                    X(:, (i+3)/4) = [cos(2*pi*10/16); sin(2*pi*10/16)];
                else
                    % 1011 => X11
                    X(:, (i+3)/4) = [cos(2*pi*11/16); sin(2*pi*11/16)];
                end
            end
        end
    end
end
```

```

        if bits(4) == 0
            % 1110 => X15
            X(:, (i+3)/4) = [cos(2*pi*15/16); sin(2*pi*15/16)];
        else
            % 1111 => X14
            X(:, (i+3)/4) = [cos(2*pi*14/16); sin(2*pi*14/16)];
        end
    end
else %bit 2 = 0
    if bits(3) == 0
        if bits(4) == 0
            % 1000 => X11
            X(:, (i+3)/4) = [cos(2*pi*11/16); sin(2*pi*11/16)];
        else
            % 1001 => X10
            X(:, (i+3)/4) = [cos(2*pi*10/16); sin(2*pi*10/16)];
        end
    else %bit 3 = 1
        if bits(4) == 0
            % 1010 => X8
            X(:, (i+3)/4) = [cos(2*pi*8/16); sin(2*pi*8/16)];
        else
            % 1011 => X9
            X(:, (i+3)/4) = [cos(2*pi*9/16); sin(2*pi*9/16)];
        end
    end
end
end
end
end
end

```

3.

(5) Να περάσετε τις ακολουθίες $\{X_{i,n}\}$ και $\{X_{q,n}\}$ από τα SRRC φίλτρα μορφοποίησης και υποθέτοντας περίοδο συμβόλου $T = 10^{-2}$ sec, over = 10, $T_s = T / \text{over}$, να σχηματίσετε και να σχεδιάσετε τις κυματομορφές εξόδου (να θέσετε το σωστό άξονα χρόνου), και τα περιοδογράμμά τους.

Αρχικά δημιουργήσαμε το srcc σήμα με παραμέτρους $T=10^{-2}$, over = 10, $\alpha = 0.5$, $A=4$. Έπειτα δημιουργήσαμε τα σήματα $X_{i,\delta}(t)$, $X_{q,\delta}(t)$ και έπειτα προσομοιώσαμε το φίλτρο κάνοντας συνέλιξη του srcc σήματος με τα παραπάνω σήματα. Τέλος υπολογίσαμε τα

περιοδογράμματα των σημάτων εξόδου με τον θεωρητικό τύπο : $P_X(F) = \frac{|F[X(t)]|^2}{T_{total}}$.

Έχουμε τον εξής κώδικα:

```

%%3
%%Plot the output of srcc filters with inputs Xi = X(1,:) , Xq = X(2,:)

T = 10^-2; %%symbol period
over = 10;
Ts = T/over; %%sampling period
A = 4;
a = 0.5;
%%create the srcc filter :

```

```

[fi,t] = srsrc_pulse(T, over, A, a);

Xi_delta = 1/Ts * upsample(X(1,:) , over);
Xq_delta = 1/Ts * upsample(X(2,:),over);

%time for X_delta signals:
t_X= 0 : Ts : N*T - Ts;

%time for the output of the filters:
t_conv = min(t)+min(t_X) : Ts : max(t)+max(t_X);

%output of the up filter
Xi_t = conv(fi,Xi_delta)*Ts;

%output of the down filter
Xq_t = conv(fi,Xq_delta)*Ts;

%Plot the outputs:
figure(1);
plot(t_conv,Xi_t);
grid on;
title('XI');
figure(2);
plot(t_conv,Xq_t);
grid on;
title('XQ');

%%Compute and plot the periodograms of XI , XQ:

Fs = 1/Ts;

Nf = 2048; % number of samples for the fourier signal

DT = Fs / Nf;

%create F axis
F = -Fs/2 : DT : (Fs/2) - DT;

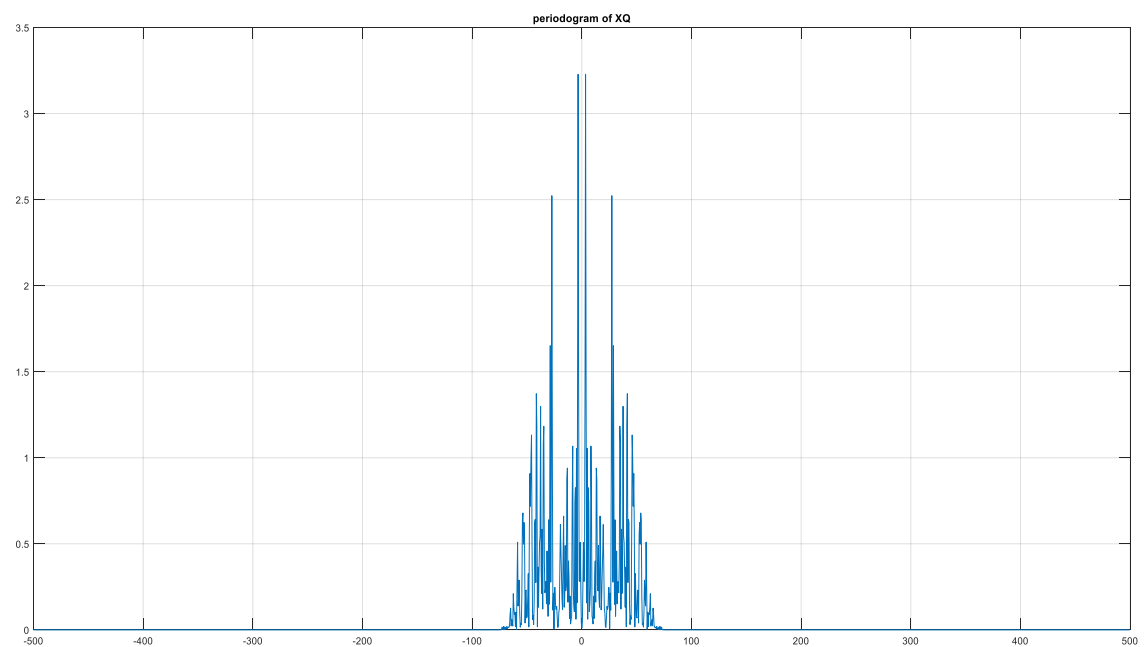
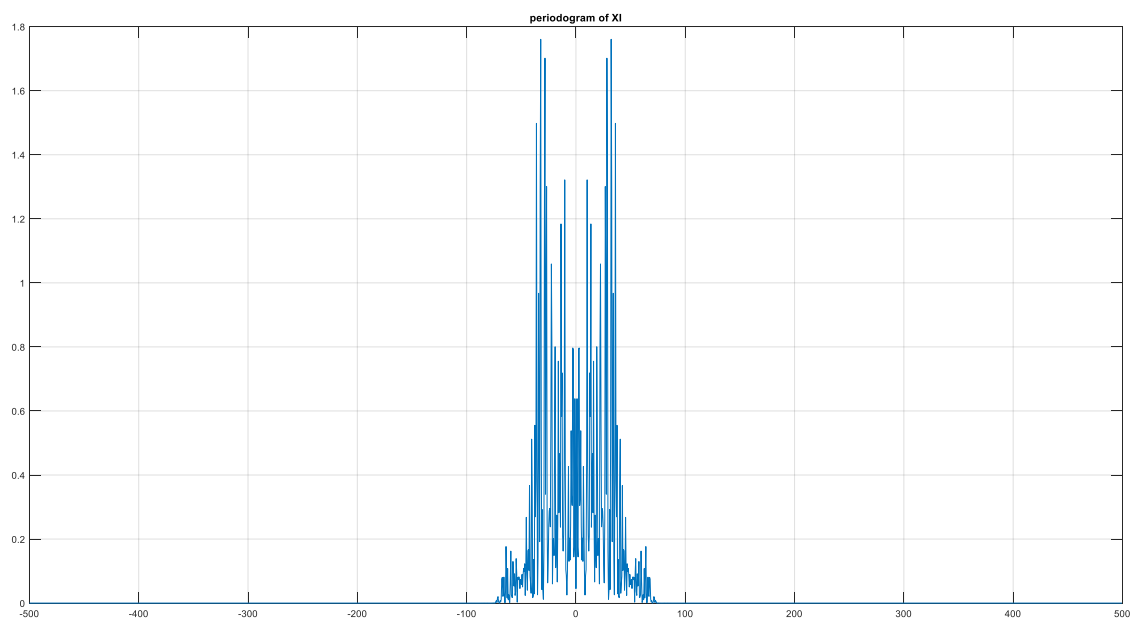
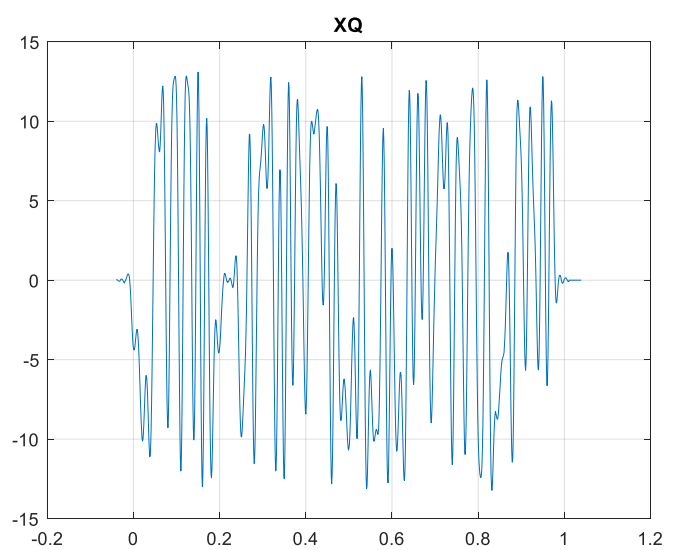
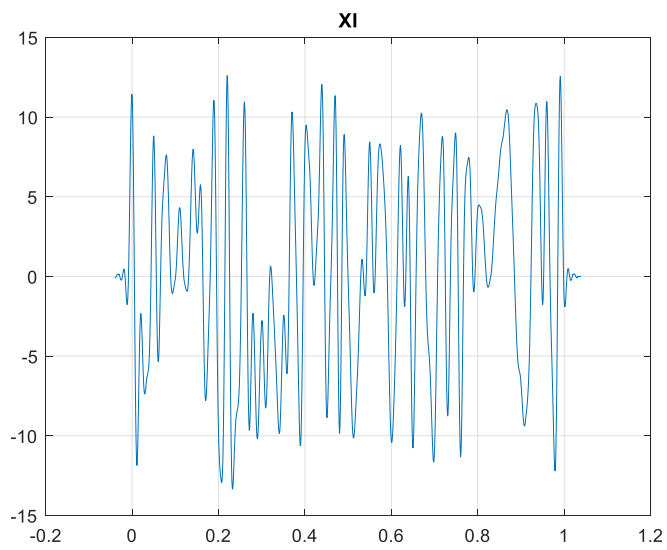
%fourier transform of XI
FXi = fftshift(fft(Xi_t,Nf))*Ts;

Ttotal = t_conv(end) - t_conv(1);
%PxI
PXin= (abs(FXi).^2) / Ttotal;
figure(3);
plot(F,PXin);
grid on;
title('periodogram of XI');

%fourier transform of XQ
FXq=fftshift(fft(Xq_t,Nf))*Ts;
%PxQ,n(F)
PXqn= (abs(FXq).^2) / Ttotal;
figure(4);
plot(F,PXqn);
grid on;
title('periodogram of XQ');

```

Και έχουμε τις παρακάτω κυματομορφές:



4.

(5) Να πολλαπλασιάσετε κατάλληλα τις εξόδους των φίλτρων με φορείς συχνότητας $F_0 = 200$ Hz και να σχεδιάσετε τις κυματομορφές που προκύπτουν, $X_I(t)$ και $X_Q(t)$, καθώς και τα αντίστοιχα περιοδογράμματα. Τι παρατηρείτε;

Στη συνέχεια παίρνοντας τα σήματα εξόδου των φίλτρων, πολλαπλασιάζουμε με τον αντίστοιχο φορέα για να μετατοπιστούν στην σωστή συχνότητα. Έπειτα τα σχεδιάζουμε, και υπολογίζουμε τα περιοδογράμματα τους.

Ο κώδικας:

```
%%4  
%%multiply the output of the filters with the carrier
```

```
F0 = 200 ; %carrier
```

```
%Multiply with the carrier:
```

```
Xi_cos = 2 * Xi_t.* cos(2*pi*F0*t_conv);  
Xq_sin = Xq_t*(-2).* sin(2*pi*F0*t_conv);
```

```
%%Plot the the signals:
```

```
figure(5);  
plot(t_conv,Xi_cos);  
grid on;  
title('Xi(t)*2cos(2πF0t)');
```

```
figure(6);  
plot(t_conv,Xq_sin);  
grid on;  
title('Xq(t)*-2sin(2πF0t)');
```

```
%%Plot the periodograms of these signals:
```

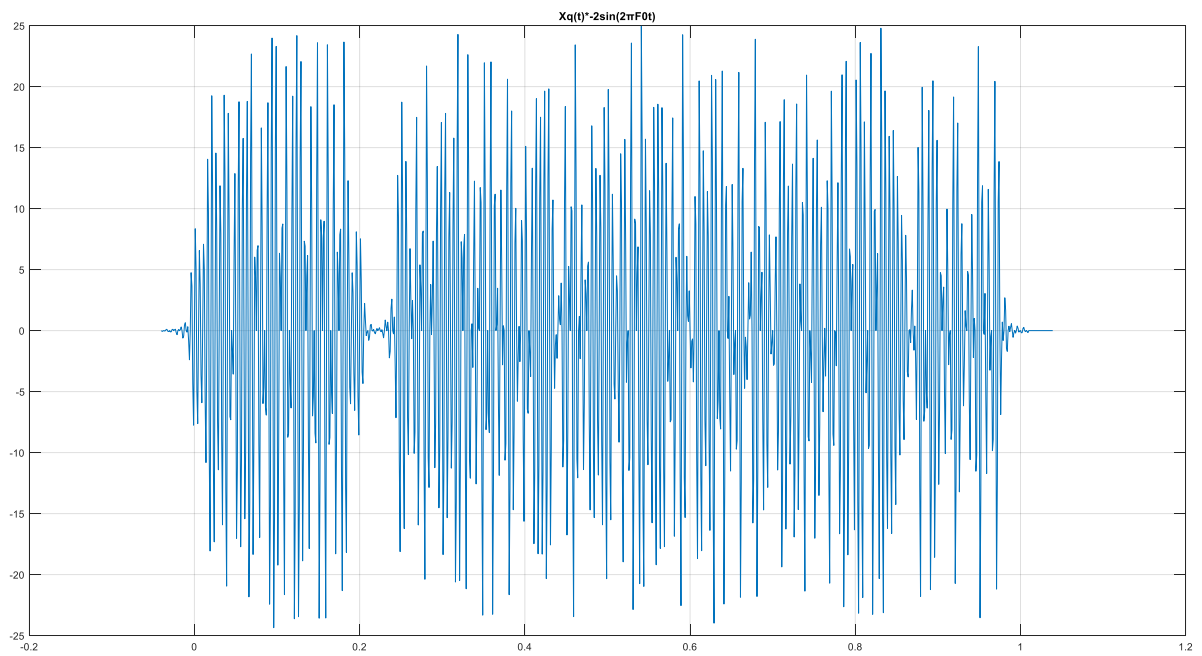
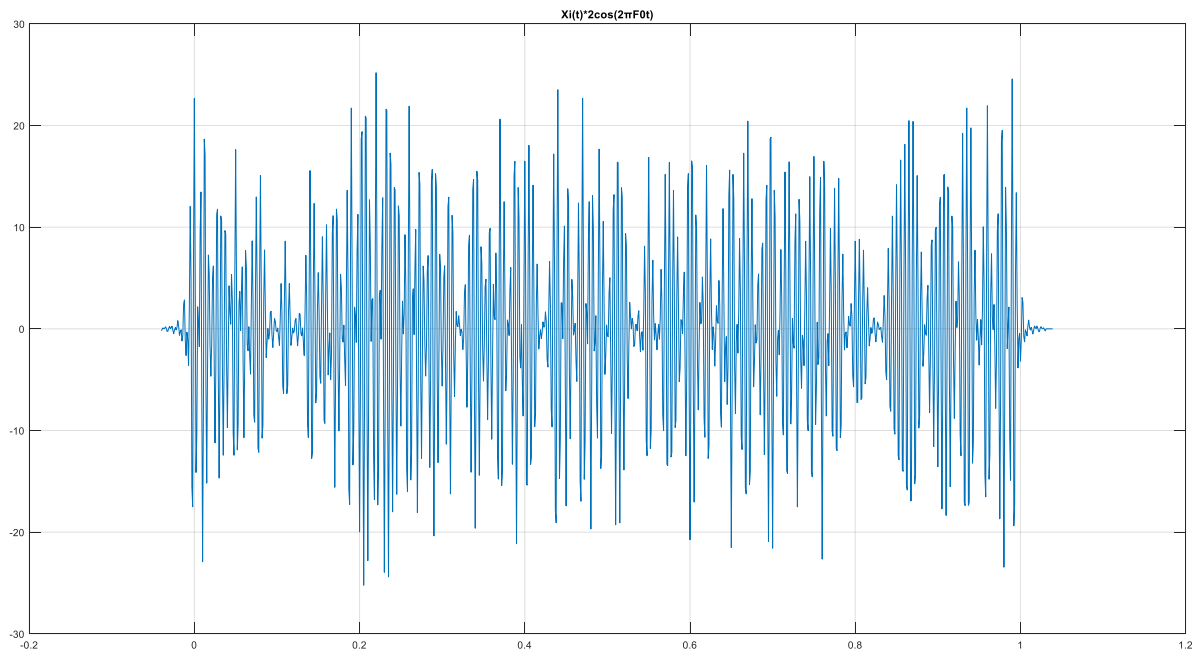
```
FXi_cos = fftshift(fft(Xi_cos,Nf)) * Ts;
```

```
%PxIcos  
%same Ttotal as Xi without cos  
PXi_cos= (abs(FXi_cos).^2) / Ttotal;  
figure(7);  
plot(F,PXi_cos);  
grid on;  
title('periodogram of Xicos');
```

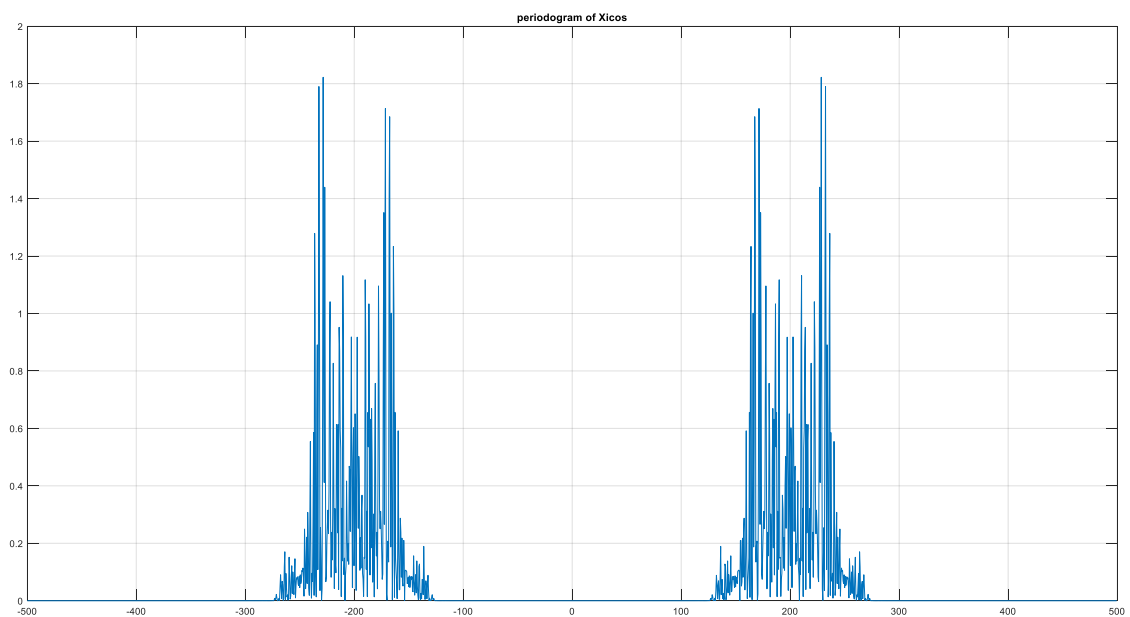
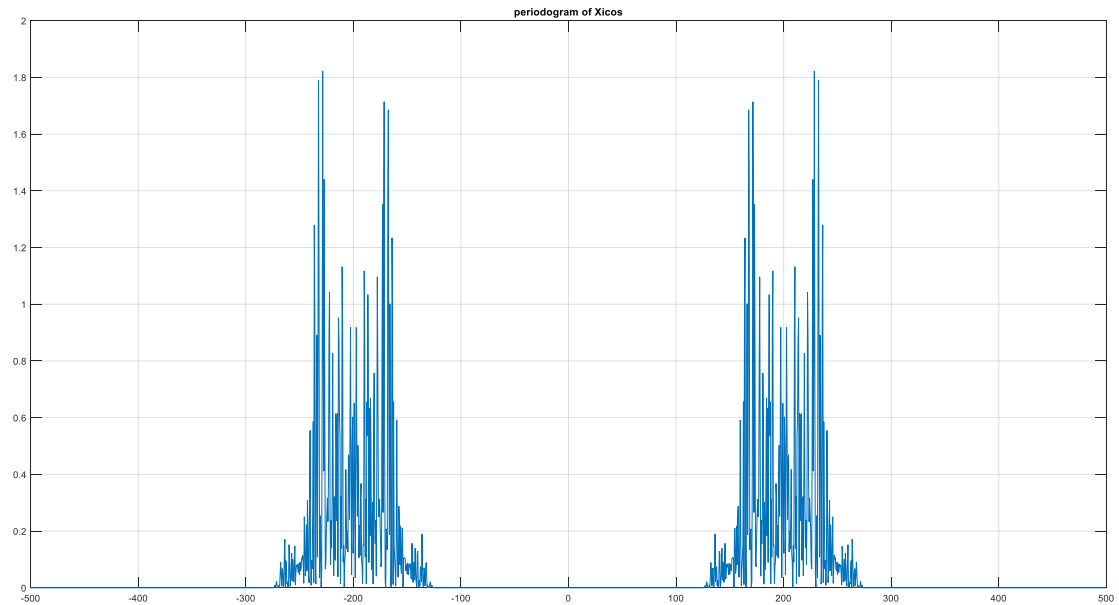
```
FXq_sin = fftshift(fft(Xq_sin,Nf)) * Ts;
```

```
%PxQsin  
PXq_sin= (abs(FXq_sin).^2) / Ttotal;  
figure(8);  
plot(F,PXq_sin);  
grid on;  
title('periodogram of Xqsin');
```


Έχουμε τις εξής κυματομορφές:



Και τα περιοδογράμματα τους:



Παρατηρήσεις:

Παρατηρούμε ότι και τα δύο περιοδογράμματα έχουν μετακινηθεί στις συχνότητες του φορέα -200 Hz , 200 Hz κάτι το οποίο είναι αναμενόμενο λόγω της ιδιότητας του μετασχηματισμού Fourier.Επίσης για τα σήματα πολλαπλασιασμένα με τον φορέα , παρατηρούμε πως ακολουθεί το περίγραμμα του αρχικού σήματος αλλά με ripples.

5.

(5) Να σχηματίσετε και να σχεδιάσετε την είσοδο του καναλιού, $X(t)$, και το περιοδόγραμμά της. Τι παρατηρείτε;

Προσθέτοντας τα δύο σήματα πολλαπλασιασμένα με φορέα παίρνουμε την έξοδο του πομπού, και την σχεδιάζουμε.

%%5

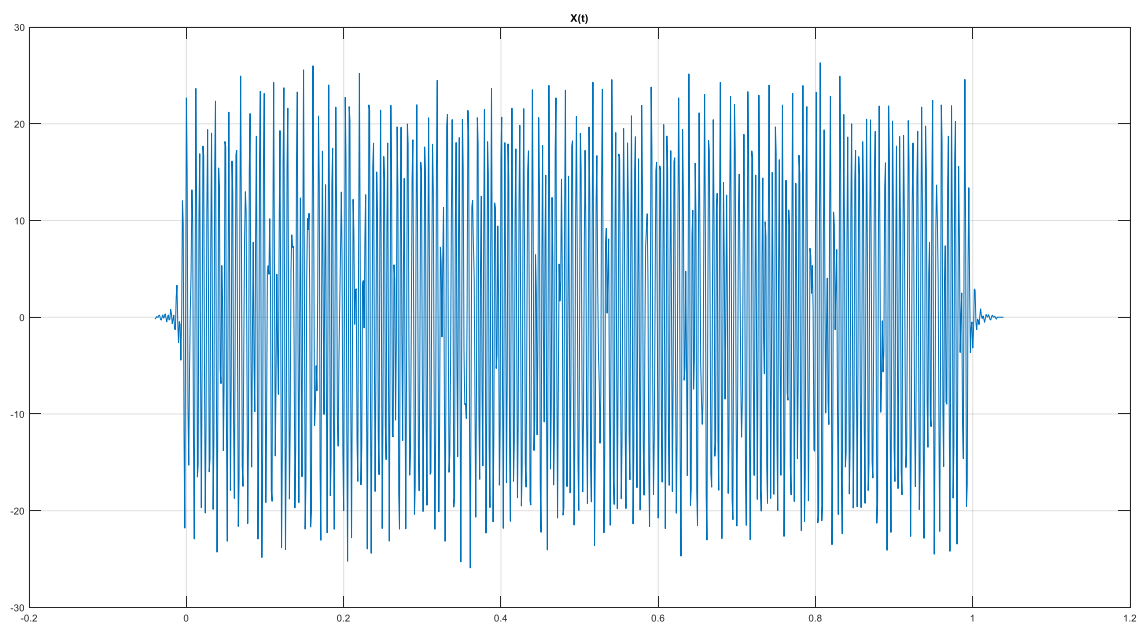
%Plot the input of the channel, $X(t)$ and it's periodogram

```
X_t = Xi_cos + Xq_sin;
```

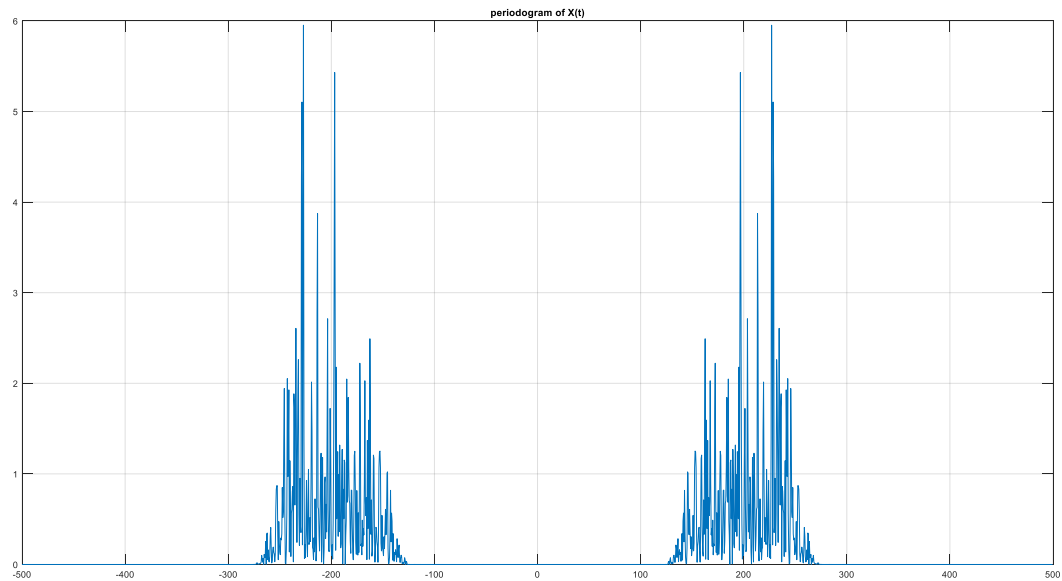
```
figure(9);  
plot(t_conv,X_t);  
grid on;  
title('X(t)');
```

```
FX_t = fftshift(fft(X_t,Nf)) * Ts;  
%PxQ  
PX_t= (abs(FX_t).^2) / Ttotal;  
figure(10);  
plot(F,PX_t);  
grid on;  
title('periodogram of X(t)');
```

Έχουμε:



Και το περιοδόγραμμα του:



Παρατηρήσεις:

Παρατηρούμε πως το περιοδόγραμμα του $X(t)$ αποτελεί το άθροισμα των 2 επιμέρους περιοδογραμάτων, κάτι το οποίο είναι αναμενόμενο λόγω γραμμικότητας του μετασχηματισμού Fourier.

7.

(5) Στην έξοδο του καναλιού, να προσθέσετε λευκό Gaussian θόρυβο $W(t)$ με διασπορά ίση με $\sigma_w^2 = \frac{1}{T_s \cdot 10^{\frac{SNR_{dB}}{10}}}$ (ενδεικτικά $SNR_{dB} = 10$ ή 20) λαμβάνοντας την ενθόρυβη κυματομορφή $Y(t) = X(t) + W(t)$.

Αρχικά δημιουργήσαμε τον λευκό θόρυβο W χρησιμοποιώντας την συνάρτηση `randn` του MATLAB πολλαπλασιάζοντας την με το σ_w ώστε να πάρουμε κανονική κατανομή με μέση τιμή 0 και διασπορά σ_w^2 . Έπειτα προσθέτουμε το σήμα αυτό στο $X(t)$ και παίρνουμε την είσοδο του δέκτη $Y(t)$.

Ο αντίστοιχος Κώδικας:

```
%%7
%%Add gaussian noise to X(t):

SNRdB = 10;
%σw^2
sigmaw = 1/(Ts * 10 ^ (SNRdB/10));
%σN^2
sigman = Ts * sigmaw/2;
```

```
%W(t)
%W(t) is gaussian with mean = 0 and sigma = sigmaw
W_t = sqrt(sigmaw) * randn(1,length(X_t));

%Y(t)
Y_t = X_t + W_t;
```

8.

(5) Να πολλαπλασιάσετε την ενθόρυβη κυματομορφή $Y(t)$ στο δέκτη με τους κατάλληλους φορείς και να σχεδιάσετε τις κυματομορφές που προκύπτουν και τα περιοδογράματά τους. Τι παρατηρείτε;

Παίρνοντας την είσοδο $Y(t)$ και πολλαπλασιάζοντας τη με τον κατάλληλο φορέα, πήραμε τις κυματομορφές και υπολογίσαμε τα περιοδογράματά τους.

Ο αντίστοιχος Κώδικας:

```
%%8

Y1_t = Y_t.* cos(2*pi*F0.*t_conv);
Y2_t = Y_t.* -sin(2*pi*F0.*t_conv);

figure(11);
plot(t_conv,Y1_t);
grid on;
title('Y(t)*cos(2πF0t) = Y1(t)');

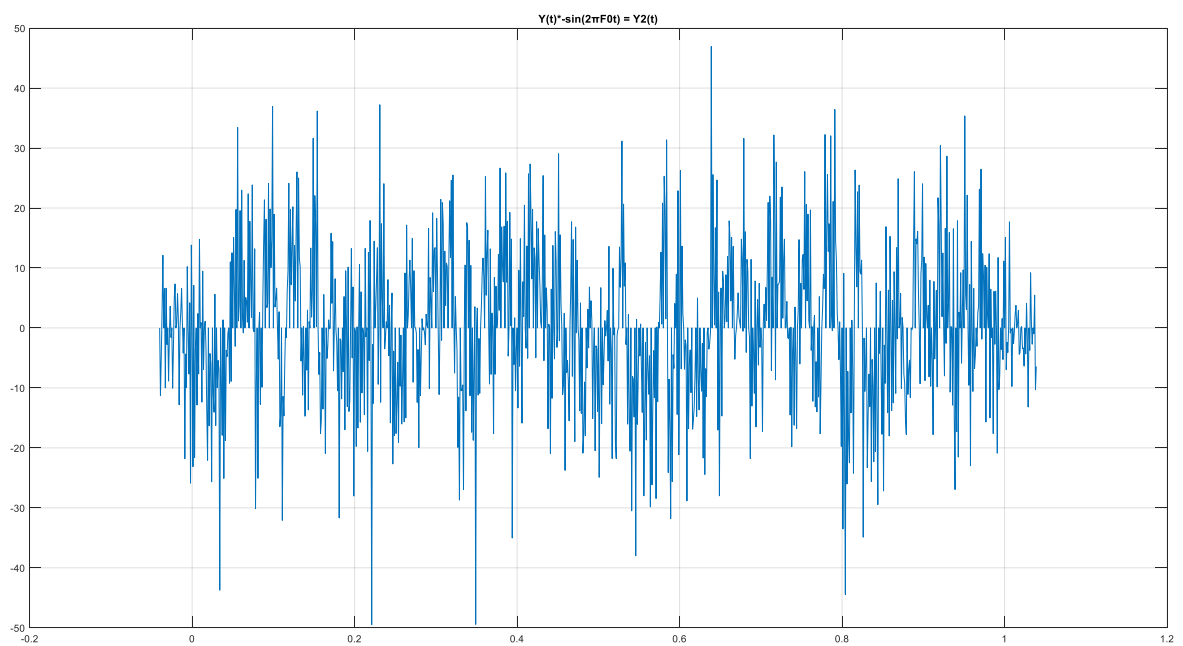
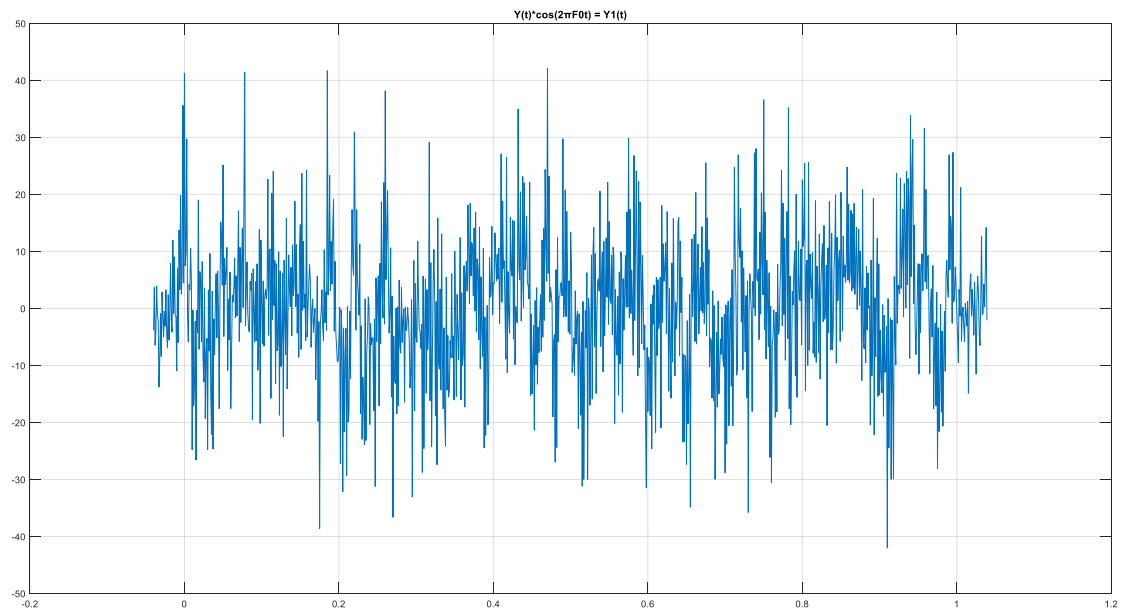
figure(12);
plot(t_conv,Y2_t);
grid on;
title('Y(t)*-sin(2πF0t) = Y2(t)');

FY1_t = fftshift(fft(Y1_t,Nf)) * Ts;
%again we have same Ttotal

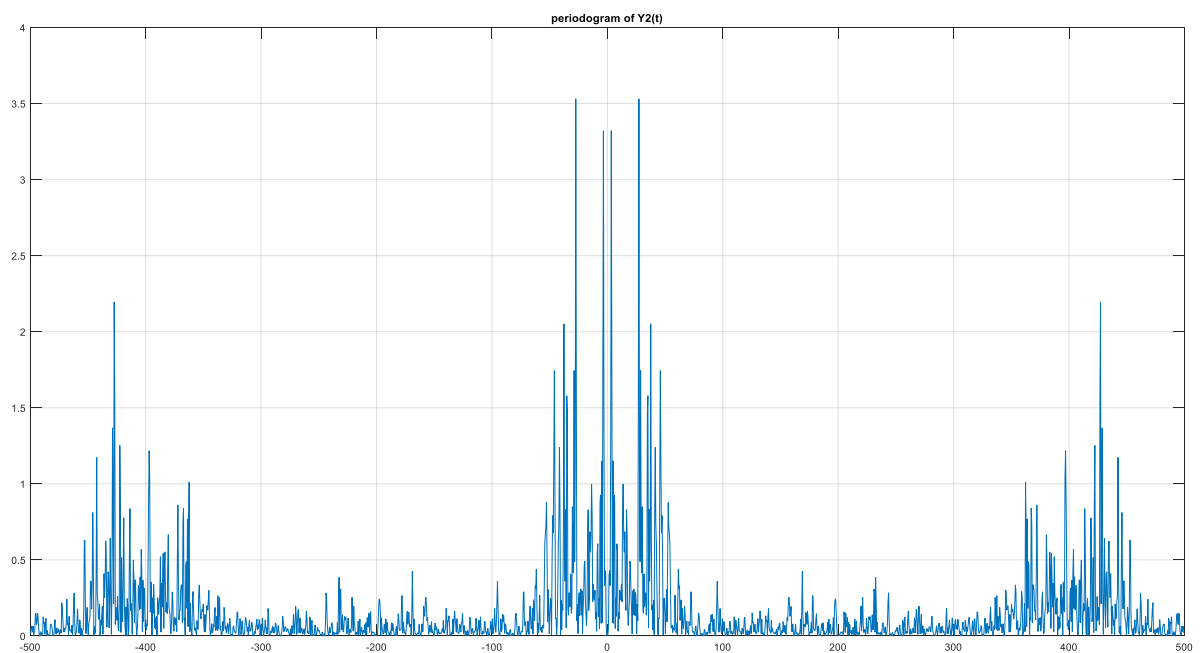
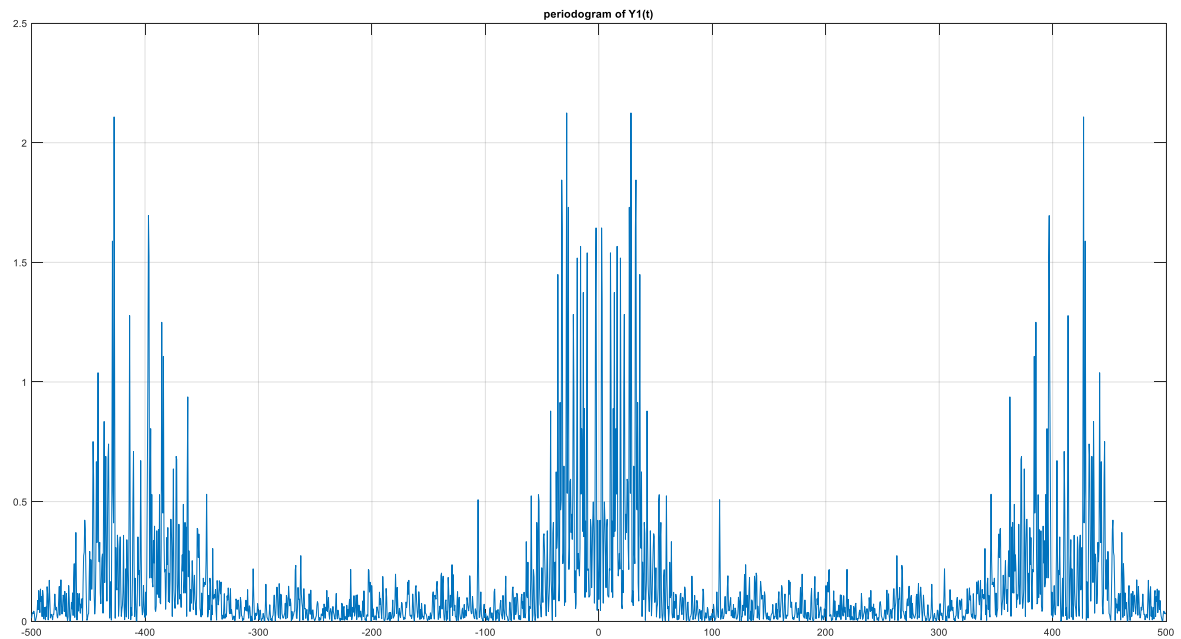
%PYI
PY1_t= (abs(FY1_t).^2)/Ttotal;
figure(13);
plot(F,PY1_t);
grid on;
title('periodogram of Y1(t)');
FY2_t = fftshift(fft(Y2_t,Nf)) * Ts;

%PYQ
PY2_t= (abs(FY2_t).^2) / Ttotal;
figure(14);
plot(F,PY2_t);
grid on;
title('periodogram of Y2(t)');
```

Και οι κυματομορφές:



Και τα περιοδογράμματα τους:



Παρατηρήσεις:

Λόγω της ιδιότητας του μετασχηματισμού fourier ο πολλαπλασιασμός με τον φορέα ξανά , μετατοπίζει τους λοβούς κατά $\pm 200\text{Hz}$ και άρα έχουμε κύριους λοβούς στο $\pm 400\text{Hz}$ και στο 0Hz στα περιοδογράμματα.

9.

(5) Να περάσετε τις κυματομορφές που υπολογίσατε στο προηγούμενο βήμα από τα προσαρμοσμένα φίλτρα. Να σχεδιάσετε τις κυματομορφές που προκύπτουν και τα περιοδογράμμά τους (να θέσετε το σωστό άξονα χρόνου). Τι παρατηρείτε;

Έπειτα περνάμε τα σήματα αυτά από τα αντίστοιχα φίλτρα , κάνοντας συνέλιξη μεταξύ τους και σχεδιάζουμε ξανά τις ζητούμενες κυματομορφές και περιοδογράμματα.

Ο αντίστοιχος Κώδικας:

```
%%9
%plot the output of the srrc filters:

Y1_tafterFilter = conv(Y1_t,fi) * Ts;
Y2_tafterFilter = conv(Y2_t,fi) * Ts;

t_convfiltered = min(t) + min(t_conv): Ts : max(t) + max(t_conv);
figure(15);
plot(t_convfiltered,Y1_tafterFilter);
grid on;
title('filtered Y1(t)');

figure(16);
plot(t_convfiltered,Y2_tafterFilter);
grid on;
title('filtered Y2(t)');

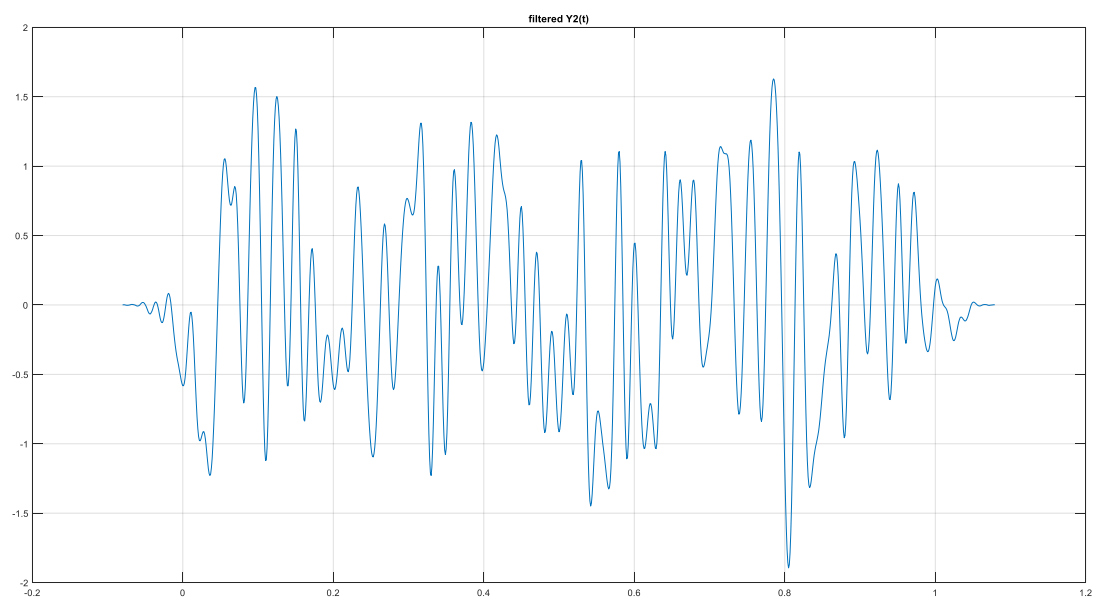
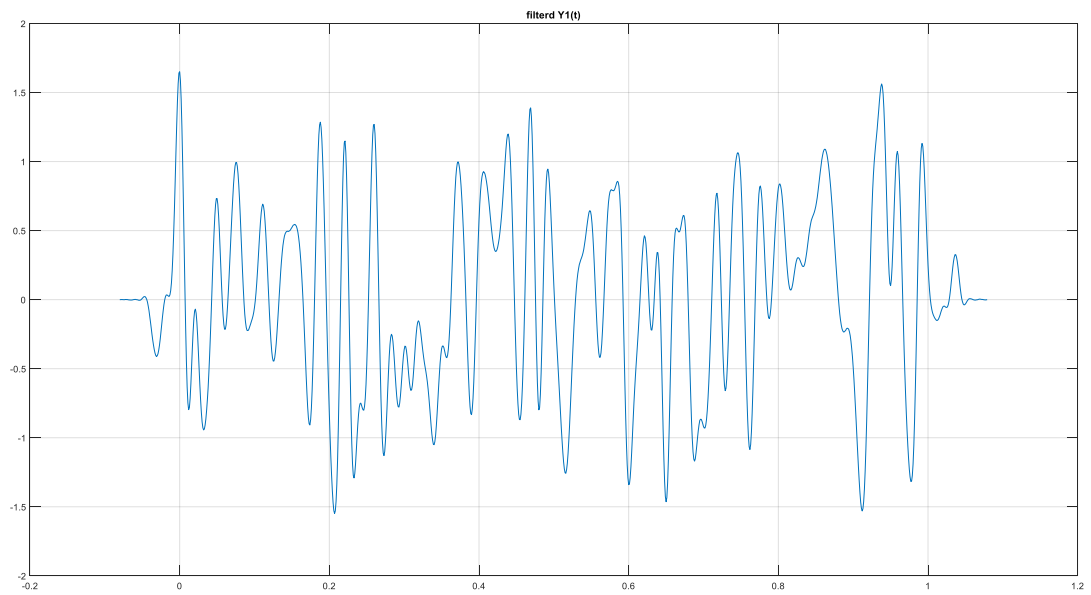
%Plot their periodograms:

FfY1_t = fftshift(fft(Y1_tafterFilter,Nf)) * Ts;

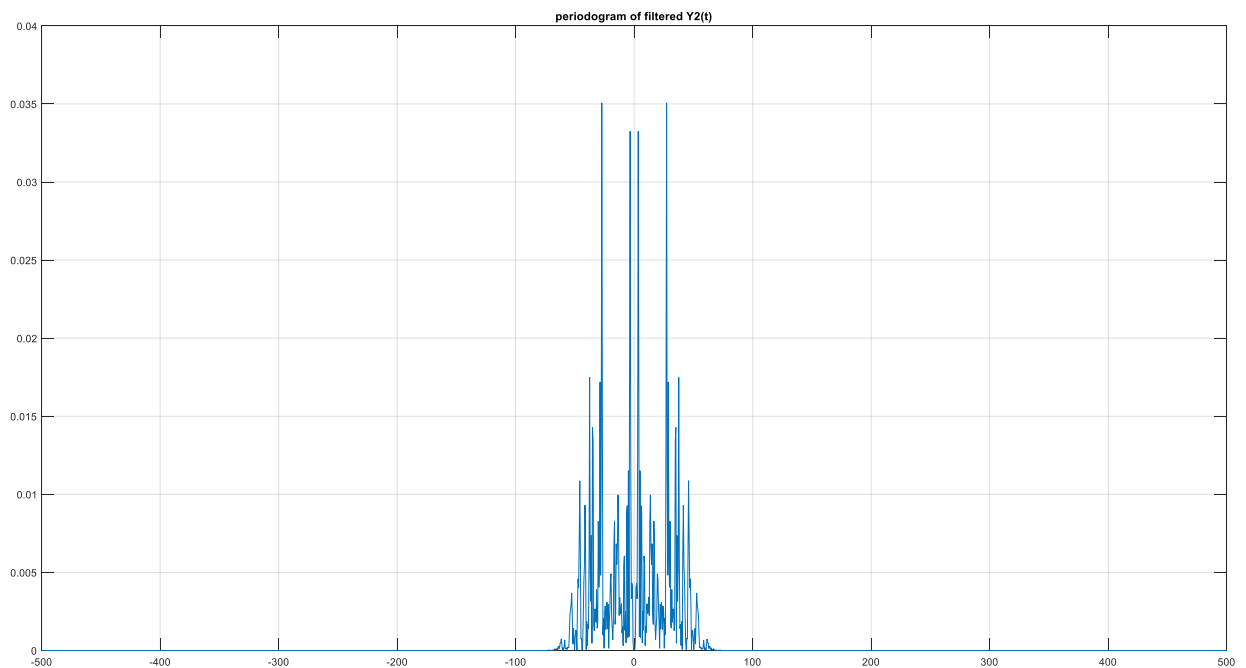
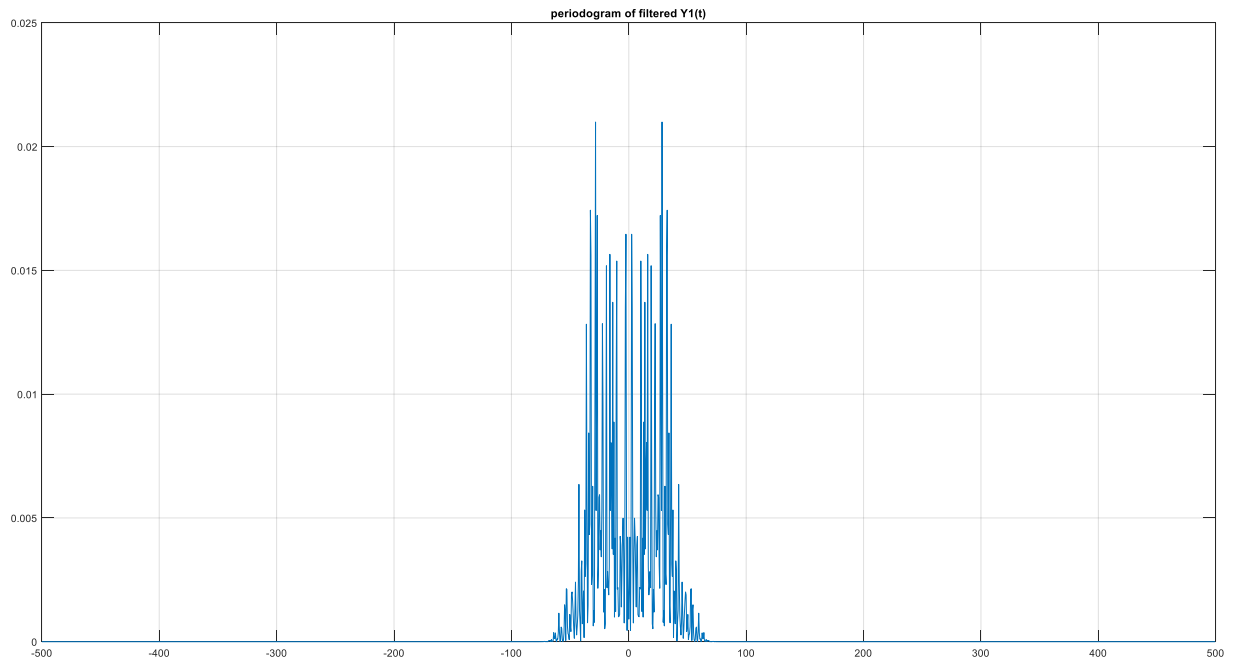
PfY1_t = (abs(FfY1_t).^2) / Ttotal;
figure(17);
plot(F,PfY1_t);
grid on;
title('periodogram of filtered Y1(t)');
FfY2_t = fftshift(fft(Y2_tafterFilter,Nf)) * Ts;

PfY2_t = (abs(FfY2_t).^2) / Ttotal;
figure(18);
plot(F,PfY2_t);
grid on;
title('periodogram of filtered Y2(t)');
```


Και οι κυματομορφές τους:



Και τα περιοδογράματά τους:



Παρατηρήσεις:

Παρατηρούμε πως τα φίλτρα στο δέκτη αποκόπτουν τις συνιστώσες οι οποίες έχουν συχνότητες γύρω από τη συχνότητα $2F_0$, δηλαδή, δρουν σαν κατωπερατά (lowpass) φίλτρα, δρουν ως προσαρμοσμένα φίλτρα στις κυματομορφές βασικής ζώνης $X_i(t)$ και $X_Q(t)$. Τέλος παρατηρούμε μια αρκετά μεγάλη μείωση στο πλάτος των κύριων λοβών σε σχέση με αυτά των αρχικών περιοδογραμάτων X_i, X_Q παρά που έχουν ίδια μορφή.

10.

(5) Να δειγματοληπτήσετε την έξοδο των προσαρμοσμένων φίλτρων τις κατάλληλες χρονικές στιγμές και να σχεδιάσετε την ακολουθία εξόδου Y χρησιμοποιώντας την εντολή `scatterplot`.

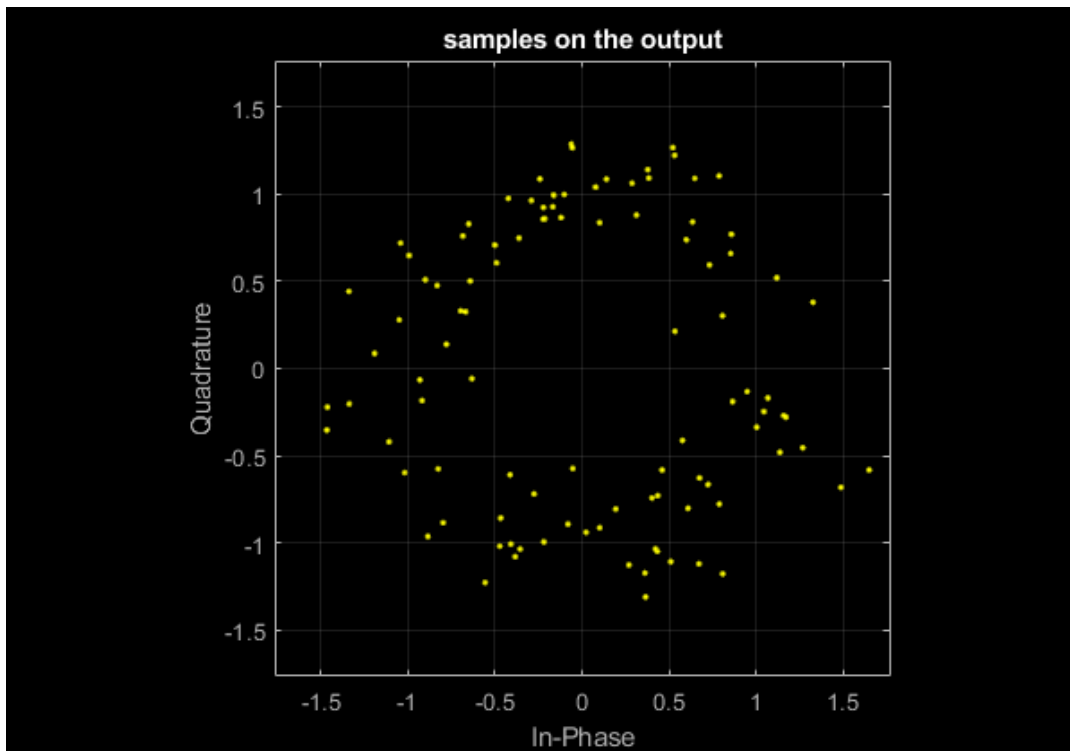
Κατά το φιλτράρισμα με το `ssrc` φίλτρο παρατηρούμε πως προστίθενται 80 δείγματα στην αρχή και στο τέλος του σήματος, των οποίων η τιμή είναι αρκετά μικρή. Έτσι παραλείπουμε τα αρχικά και τα τελικά 80 ($2 \cdot A \cdot \text{over}$) δείγματα και έχουμε βήμα `over` ώστε να κάνουμε `downsample` και να πάρουμε το σωστό δείγμα, αφού στην αρχή κάναμε `upsample` κατά `over`.

Ο αντίστοιχος κώδικας:

```
%%10
i=1;
for j=2*A*over+1:over:length(t_convfiltered)-2*A*over
    Y(i,1)=Y1_tafterFilter(j);
    Y(i,2)=Y2_tafterFilter(j);
    i=i+1;
end

scatterplot(Y);
grid on;
title('samples on the output');
```

Και έχουμε:



Παρατηρήσεις:

Παρατηρούμε πως τα δείγματα εξόδου έχουν κυκλική δομή με ακτίνα $A=1$, απλά με την προσθήκη θορύβου. Αυτό είναι κάτι αναμενόμενο λόγω του 16PSK modulation.

11.

Να γράψετε συνάρτηση `function [est X, est bit seq] = detect PSK 16(Y)` η οποία
(α) (10) χρησιμοποιεί τον κανόνα εγγύτερου γείτονα και αποφασίζει για την ακολουθία εισόδου 16-PSK σύμβολο-προς -σύμβολο,

(β) (10) χρησιμοποιεί την αντίστροφη απεικόνιση Gray, δηλαδή, από σύμβολα σε τετράδες bits, και από την εκτιμώμενη ακολουθία συμβόλων εισόδου υπολογίζει την εκτιμώμενη δυαδική ακολουθία εισόδου.

Αρχικά υπολογίζουμε την ευκλείδια απόσταση του κάθε σημείου του διανύσματος Y με όλα τα σημεία του αλφαβήτου 16PSK. Στη συνέχεια εκτιμούμε πως το σύμβολο αυτό ανήκει στην κατηγορία η οποία έχει την μικρότερη ευκλείδια απόσταση. Έπειτα κάνουμε την αντίστροφη διαδικασία για αποκωδικοποίηση των συμβόλων σε bits χρησιμοποιώντας το ίδιο gray encoding.

Ο αντίστοιχος Κώδικας:

```
function [est_X, est_bit_seq] = detect_PSK_16_gray(Y)
    for j = 1:length(Y)
        for m = 1:16
            nearest_neighbor(m) = sqrt(((Y(j,1) - cos((2*pi*(m-1))/16))^2)
+ ((Y(j,2) - sin((2*pi*(m-1))/16))^2));
        end
        minNeighbor = min(nearest_neighbor);
        for i = 1:16
            if minNeighbor == nearest_neighbor(i)
                k = i;
            end
        end
        est_X(1,j) = cos(2*pi*(k-1)/16);
        est_X(2,j) = sin(2*pi*(k-1)/16);
    end

    j = 1;
    for i = 1:4:4*length(Y)
        if est_X(1,j) == cos(2*pi*0/16) && est_X(2,j) == sin(2*pi*0/16)
            % 0000 => X0
            est_bit_seq(i:i+3,1) = [0; 0; 0; 0];

        elseif est_X(1,j) == cos(2*pi*1/16) && est_X(2,j) == sin(2*pi*1/16)
            % 0001 => X1
            est_bit_seq(i:i+3,1) = [0; 0; 0; 1];

        elseif est_X(1,j) == cos(2*pi*2/16) && est_X(2,j) == sin(2*pi*2/16)
            % 0011 => X2
            est_bit_seq(i:i+3,1) = [0; 0; 1; 1];

        elseif est_X(1,j) == cos(2*pi*3/16) && est_X(2,j) == sin(2*pi*3/16)
```

```

        % 0010 => X3
        est_bit_seq(i:i+3,1) = [0; 0; 1; 0];

    elseif est_X(1,j) == cos(2*pi*4/16) && est_X(2,j) == sin(2*pi*4/16)
        % 0110 => X4
        est_bit_seq(i:i+3,1) = [0; 1; 1; 0];

    elseif est_X(1,j) == cos(2*pi*5/16) && est_X(2,j) == sin(2*pi*5/16)
        % 0111 => X5
        est_bit_seq(i:i+3,1) = [0; 1; 1; 1];

    elseif est_X(1,j) == cos(2*pi*6/16) && est_X(2,j) == sin(2*pi*6/16)
        % 0101 => X6
        est_bit_seq(i:i+3,1) = [0; 1; 0; 1];

    elseif est_X(1,j) == cos(2*pi*7/16) && est_X(2,j) == sin(2*pi*7/16)
        % 0100 => X7
        est_bit_seq(i:i+3,1) = [0; 1; 0; 0];

    elseif est_X(1,j) == cos(2*pi*8/16) && est_X(2,j) == sin(2*pi*8/16)
        % 1010 => X8
        est_bit_seq(i:i+3,1) = [1; 0; 1; 0];

    elseif est_X(1,j) == cos(2*pi*9/16) && est_X(2,j) == sin(2*pi*9/16)
        % 1011 => X9
        est_bit_seq(i:i+3,1) = [1; 0; 1; 1];

    elseif est_X(1,j) == cos(2*pi*10/16) && est_X(2,j) ==
sin(2*pi*10/16)
        % 1001 => X10
        est_bit_seq(i:i+3,1) = [1; 0; 0; 1];

    elseif est_X(1,j) == cos(2*pi*11/16) && est_X(2,j) ==
sin(2*pi*11/16)
        % 1000 => X11
        est_bit_seq(i:i+3,1) = [1; 0; 0; 0];

    elseif est_X(1,j) == cos(2*pi*12/16) && est_X(2,j) ==
sin(2*pi*12/16)
        % 1100 => X12
        est_bit_seq(i:i+3,1) = [1; 1; 0; 0];

    elseif est_X(1,j) == cos(2*pi*13/16) && est_X(2,j) ==
sin(2*pi*13/16)
        % 1101 => X13
        est_bit_seq(i:i+3,1) = [1; 1; 0; 1];

    elseif est_X(1,j) == cos(2*pi*14/16) && est_X(2,j) ==
sin(2*pi*14/16)
        % 1111 => X14
        est_bit_seq(i:i+3,1) = [1; 1; 1; 1];

    else
        % 1110 => X15
        est_bit_seq(i:i+3,1) = [1; 1; 1; 0];
    end
    j = j + 1;
end
end

```

12.

(10) Να γράψετε συνάρτηση `function num_of_symbol_errors = symbol_errors(est_X, X)` η οποία υπολογίζει το πλήθος των σφαλμάτων εκτίμησης συμβόλου.

Η συνάρτηση αυτή ελέγχει ένα προς ένα τα εκτιμώμενα σύμβολα (διάνυσμα στήλης) με τα πραγματικά σύμβολα. Αν υπάρξει έστω και ένα στοιχείο διανύσματος διαφορετικό μετριέται ως σφάλμα.

Ο κώδικας της Συνάρτησης:

```
function num_of_symbol_errors = symbol_errors(est_X,X)
    num_of_symbol_errors=0;
    for i=1:length(X)
        if est_X(i)~=X(i)
            % if est_X(1,i)~=X(1,i) || est_X(2,i)~=X(2,i)
                num_of_symbol_errors=num_of_symbol_errors+1;
            end
        end
    end
end
```

Για την αρχική μας υλοποίηση υπήρξε σφάλμα : `num_of_symbol_errors = 36`

13.

(10) Να γράψετε συνάρτηση `function num_of_bit_errors = bit_errors(est_bit_seq, b)` η οποία υπολογίζει το πλήθος των σφαλμάτων εκτίμησης bit.

Η συνάρτηση αυτή ελέγχει ένα προς ένα τα εκτιμώμενα bit με τα πραγματικά bit. Αν υπάρξει έστω και ένα bit διαφορετικό μετριέται ως ένα σφάλμα.

```
function num_of_bit_errors = bit_errors(est_bit_seq,bit_seq)
    num_of_bit_errors=0;
    for i=1:length(bit_seq)
        if est_bit_seq(i) ~= bit_seq(i)
            num_of_bit_errors=num_of_bit_errors+1;
        end
    end
end
```

Για την αρχική μας υλοποίηση υπήρξε σφάλμα : `num_of_bit_errors = 57`

Στο δεύτερο μέρος, θα εκτιμήσουμε την πιθανότητα σφάλματος συμβόλου και bit με χρήση της μεθόδου Monte Carlo.

1.

Για $\text{SNR}_{\text{dB}} = [-2 : 2 : 24]$, να εκτιμήσετε πειραματικά την πιθανότητα σφάλματος απόφασης συμβόλου και bit επαναλαμβάνοντας τα παραπάνω βήματα K φορές (ενδεικτικά, $K = 1000$), για κάθε SNR. Η πειραματική εκτίμηση της πιθανότητας σφάλματος συμβόλου, για κάθε SNR, υπολογίζεται ως εξής:

$$P(\text{Esymbol}) = \frac{\text{συνολικό πλήθος σφαλμάτων απόφασης συμβόλου}}{\text{συνολικό πλήθος απεσταλμένων συμβόλων}}$$

ενώ η πιθανότητα σφάλματος bit εκτιμάται ως εξής:

$$P(\text{Ebit}) = \frac{\text{συνολικό πλήθος σφαλμάτων απόφασης bit}}{\text{συνολικό πλήθος απεσταλμένων bits}}$$

Η παραπάνω διαδικασία έγινε με 2 for loop, ένα για το SNR_{dB} και ένα για το πλήθος K . Έτσι υπολογίστηκε η πειραματική εκτίμηση πιθανότητας σφάλματος.

%%2o meros:

```
SNRdB = -2:2:24;
```

```
M = 16;
```

```
Pf_symbol = zeros(size(SNRdB));
```

```
Pf_bit = zeros(size(SNRdB));
```

```
for j = 1:length(SNRdB)
    sum_num_symbol_errors = 0;
    sum_num_bit_errors = 0;
    for K = 1:1000
        N = 100;
        bit_seq = (sign(randn(4 * N, 1)) + 1) / 2;
        X = bitsTo16PSK(bit_seq);
        T = 10^-2;
        over = 10;
        Ts = T / over;
        A = 4;
        a = 0.5;
        Fs = 1 / Ts;
        Nf = 2048;
        DT = Fs / Nf;
        [phi, t] = srrc_pulse(T, over, A, a);
        Xi_delta = 1 / Ts * upsample(X(1, :), over);
        Xq_delta = 1 / Ts * upsample(X(2, :), over);
        Xi_t = conv(phi, Xi_delta) * Ts;
        Xq_t = conv(phi, Xq_delta) * Ts;

        F0 = 200;
        Xi_cos = 2 * Xi_t .* cos(2 * pi * F0 * t_conv);
        Xq_sin = 2 * Xq_t .* -sin(2 * pi * F0 * t_conv);
```

```

X_t = Xi_cos + Xq_sin;
sigmaw = 1 / (Ts * (10^(SNRdB(j)/10)));
sigmansq = Ts * ((sigmaw)) / 2;
% W(t)
W_t = sqrt(sigmaw) * randn(1, length(X_t));
Y_t = X_t + W_t;
Y1_t = Y_t .* cos(2 * pi * F0 * t_conv);
Y2_t = Y_t .* -sin(2 * pi * F0 * t_conv);
filteredY1_t = conv(Y1_t, phi) * Ts;
filteredY2_t = conv(Y2_t, phi) * Ts;
l = 1;
for k=2*A*over+1:over:length(filteredY1_t)-2*A*over
    Y(l,1)=filteredY1_t(k);
    Y(l,2)=filteredY2_t(k);
    l=l+1;
end

[est_X, est_bit_seq] = detect_PSK_16_gray(Y);
num_of_symbol_errors1 = symbol_errors(est_X, X);
num_of_bit_errors1 = bit_errors(est_bit_seq, bit_seq);
sum_num_symbol_errors = sum_num_symbol_errors +
num_of_symbol_errors1;
sum_num_bit_errors = sum_num_bit_errors + num_of_bit_errors1;
end
% Calculate average symbol error probability
PEsymbol(j) = sum_num_symbol_errors / (N * K);

% Calculate average bit error probability
PEbit(j) = sum_num_bit_errors / (4 * N * K);

% Calculate theoretical symbol error probability for 16-PSK modulation
Pf_symbol(j) = 2*qfunc(1*sin(pi/M)/sqrt(sigmansq));
% Calculate theoretical bit error probability
Pf_bit(j) = Pf_symbol(j) ./ log2(M);
end

```

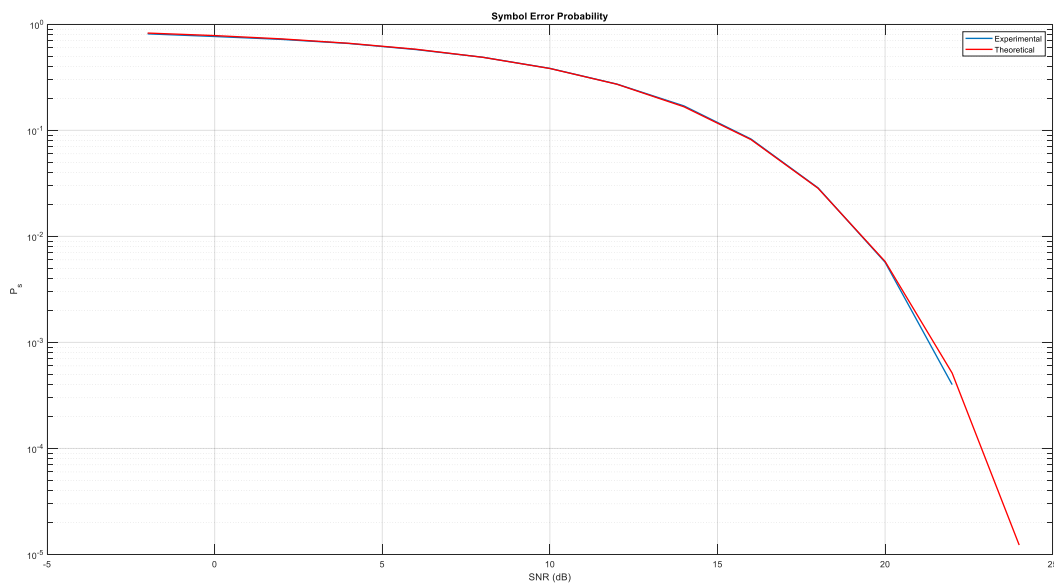

2.

(20) Να σχεδιάσετε σε semilogy την εκτιμώμενη πιθανότητα σφάλματος συμβόλου ως συνάρτηση του SNRdB. Στο ίδιο σχήμα, να σχεδιάσετε και το έξυπνο άνω φράγμα για την πιθανότητα σφάλματος συμβόλου. Τι παρατηρείτε;

Ο αντίστοιχος κώδικας:

```
%2
figure();
semilogy(SNRdB, PEsymbol, 'LineWidth', 1.5);
hold on;
semilogy(SNRdB, Pf_symbol, 'r', 'LineWidth', 1.5);
grid on;
title('Symbol Error Probability');
xlabel('SNR (dB)');
ylabel('P_s');
legend('Experimental', 'Theoretical');
```

Η κυματομορφή θεωρητικού και πειραματικού σφάλματος συμβόλου:



Παρατηρήσεις:

Παρατηρούμε πως έχουμε μία παρα πολύ καλη προσέγγιση του πειραματικού με το άνω θεωρητικό φράγμα.

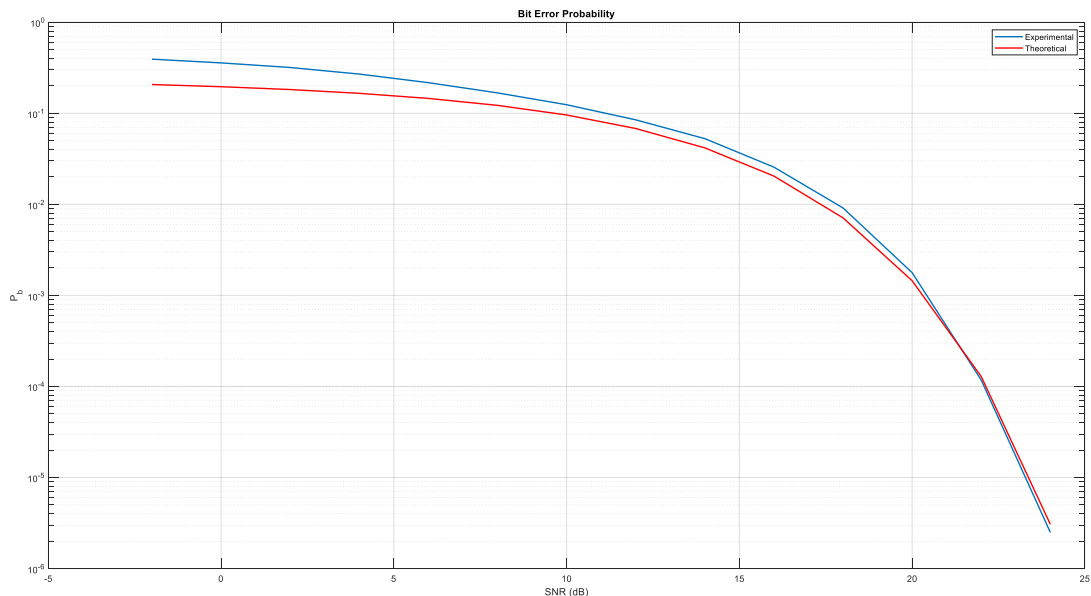
3.

(20) Να σχεδιάσετε σε semilogy την εκτιμώμενη πιθανότητα σφάλματος bit ως συνάρτηση του SNRdB. Στο ίδιο σχήμα, να σχεδιάσετε και το κάτω φράγμα για την πιθανότητα σφάλματος bit, το οποίο προκύπτει από την εκτιμώμενη πιθανότητα σφάλματος συμβόλου και την κωδικοποίηση Gray. Τι παρατηρείτε; Μπορείτε να εξηγήσετε το φαινόμενο;

Ο αντίστοιχος κώδικας:

```
%3
figure();
semilogy(SNRdB,PEbit,'LineWidth',1.5);
hold on;
semilogy(SNRdB,Pf_bit,'r','LineWidth',1.5);
grid on;
title('Bit Error Probability');
xlabel('SNR (dB)');
ylabel('P_b');
legend('Experimental','Theoretical');
```

Η κυματομορφή θεωρητικού και πειραματικού σφάλματος bit:



Παρατηρήσεις:

Παρατηρούμε ότι έχουμε μία λιγότερο καλή προσέγγιση στα χαμηλά SNRdB σε σχέση με αυτήν του σφάλματος συμβόλου. Λόγω του μεγάλου θορύβου συγκριτικά με το σήμα στα χαμηλά SNRdB έχουμε μεγαλύτερες πιθανότητα σφάλματος στα bit.